HOSTEL MANAGEMENT SYSTEM

Object-Oriented Programming (C++) Project Report

Submitted To

Sir Khawaja Badar

Submitted By

- Bilal Abid Abbasi
- Hubaib Khan
- Muqtadir Khan

Institution

Iqra University

Course

Object-Oriented Programming Lab (C++)

Submission Date

9 JUNE 2026

Abstract

The Hostel Management System is a console-based application developed in C++ using Object-Oriented Programming (OOP) concepts. The system is designed to manage hostel student records efficiently by providing facilities for student registration, searching, updating information, room allocation, fee management, and record deletion.

Traditional manual record management systems are often time-consuming and prone to errors. This application automates hostel administration tasks, improves record accuracy, and provides quick access to student information. The project demonstrates practical implementation of inheritance, polymorphism, encapsulation, classes, objects, and data management techniques in C++.

---

1. Introduction

Hostel management is an important administrative task in educational institutions. Maintaining student records manually can result in data duplication, record loss, and inefficient management.

The Hostel Management System provides a computerized solution for managing hostel records. The system allows administrators to add student information, search records, update fees and room assignments, display student details, and remove records when required.

The project has been developed using C++ and follows Object-Oriented Programming principles to ensure modularity, maintainability, and code reusability.

---

2. Problem Statement

Manual hostel management systems suffer from several limitations:

- Difficulty in maintaining student records.
- Time-consuming record searching.
- Errors during room allocation.
- Inefficient fee management.
- Increased paperwork.
- Lack of centralized information storage.

The proposed Hostel Management System aims to automate these tasks and improve overall administrative efficiency.

---

3. Objectives

The objectives of the Hostel Management System are:

3.1 Student Record Management

Store and maintain student information digitally.

3.2 Room Management

Assign and update hostel room numbers.

3.3 Fee Management

Maintain hostel fee records and update payments.

3.4 Search Facility

Quickly search student records using Student ID.

3.5 Record Maintenance

Modify and delete student records whenever required.

---

4. Scope of the Project

Included Features

- Add Student Records
- Display Student Records
- Search Student by ID
- Update Student Fee
- Change Room Number
- Delete Student Record

Future Enhancements

- File Handling for Permanent Storage

- Database Integration
- Online Fee Management
- Student Login Portal
- Web-Based Hostel Management System

---

5. Methodology

The project was developed using the Object-Oriented Programming approach.

Requirement Analysis

The following entities were identified:

- Person
- Student
- Hostel

System Design

The system was divided into classes where each class performs specific tasks.

Implementation

The project was implemented using:

- Classes and Objects
- Inheritance
- Polymorphism
- Arrays
- Input Validation

Testing

Each module was tested separately to verify correctness and functionality.

---

6. System Design

Class Structure

Person Class

Base class containing:

- Name
- Age

Student Class

Derived class inherited from Person.

Additional attributes:

- Student ID
- Father Name
- CNIC
- Contact Number
- Room Number
- Hostel Fee

Hostel Class

Manages all student records and provides operations such as:

- Add Student
- Display Students
- Search Student
- Update Fee
- Update Room
- Delete Student

---

7. Functional Requirements

Student Registration Module

- Enter student information.
- Store student details.

Search Module

- Search students using Student ID.

Fee Management Module

- Update hostel fee records.

Room Management Module

- Change room assignments.

Record Deletion Module

- Remove student records when required.

---

8. OOP Concepts Implemented

8.1 Classes and Objects

Classes used:

- Person
- Student
- Hostel

Objects are created from these classes to perform operations.

8.2 Encapsulation

Private data members are used to secure sensitive information such as:

- Student ID
- CNIC
- Contact Number
- Fee

Access is provided through public member functions.

8.3 Inheritance

Student class inherits common attributes from Person class.

Example:

Student → Person

This reduces code duplication.

8.4 Polymorphism

Runtime polymorphism is implemented using:

- Virtual Function
- Function Overriding

The display() function in Student overrides the display() function of Person.

---

9. Program Workflow

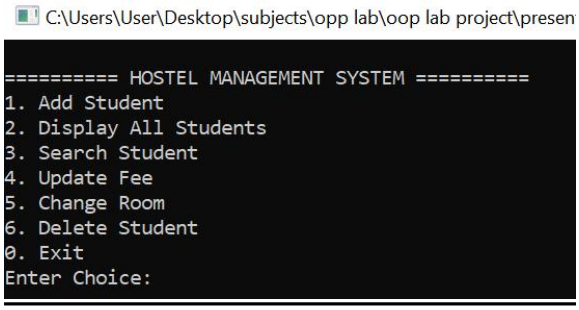
1. User launches the application.
2. Main menu appears.
3. User selects an operation.
4. Student records are added or modified.
5. Fees and room information are updated.
6. Student records are searched.
7. Records can be deleted when necessary.
8. Program exits upon user request.

---

10. Output Screens

(Insert screenshots of your program output here)

Main Menu



Add Student

C:\Users\User\Desktop\subjects\opp lab\oop lab project\presentation cod

```
========== HOSTEL MANAGEMENT SYSTEM ==========
1. Add Student
2. Display All Students
3. Search Student
4. Update Fee
5. Change Room
6. Delete Student
0. Exit
Enter Choice: 1
Enter Student Name: bilal
Enter Father Name: abid
Enter CNIC: 3740412881077
Enter Contact Number: 03710181459
Enter Student ID: 45961
Enter Age: 20
Enter Room Number: 19
Enter Hostel Fee: 20000

Student Added Successfully!
```

Search Student

```
========== HOSTEL MANAGEMENT SYSTEM ==========
1. Add Student
2. Display All Students
3. Search Student
4. Update Fee
5. Change Room
6. Delete Student
0. Exit
Enter Choice: 2

No Students Found!
```

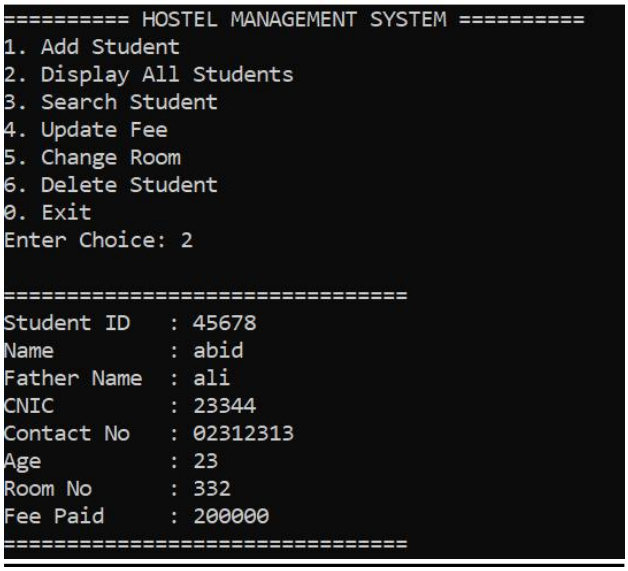
Update Fee

C:\Users\User\Desktop\subjects\opp lab\oop lab project\presentation co

```
========== HOSTEL MANAGEMENT SYSTEM ==========
1. Add Student
2. Display All Students
3. Search Student
4. Update Fee
5. Change Room
6. Delete Student
0. Exit
Enter Choice: 4
Enter Student ID: 45961

Student Not Found!
```

Display Records



.

11. Advantages of the System

- Faster record management
- Easy student searching
- Improved accuracy
- Reduced paperwork
- Better room allocation management
- Efficient fee tracking
- User-friendly interface

---

12. Limitations

- Data is stored only during program execution.
- No database integration.
- No file handling.
- Limited hostel capacity (100 students).

---

13. Conclusion

The Hostel Management System successfully demonstrates the practical implementation of Object-Oriented Programming concepts in C++. The system provides an efficient solution for managing hostel records through student registration, searching, fee management, room allocation, and record deletion.

The project improves administrative efficiency and serves as a strong example of applying inheritance, polymorphism, encapsulation, and modular programming techniques in real-world applications.

14.References

1. Object-Oriented Programming in C++
2. C++ Programming by Bjarne Stroustrup.
3. C++ Standard Library Documentation.
4. Object-Oriented Programming Course Notes.

15. Source Code

The complete C++ source code is attached in the Appendix section.

```
#include <iostream>

#include <string>

#include <limits>

using namespace std;


// Base Class

class Person

{
```

```
protected:
    string name;
    int age;

public:
    virtual void display()
    {
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
    }

    virtual ~Person() {}
};

// Derived Class
class Student : public Person
{
private:
    int studentID;
    string fatherName;
    string cnic;
    string contactNo;
    int roomNo;
    float fee;
```

```
public:
    Student()
    {
        studentID = 0;
        fatherName = "";
        cnic = "";
        contactNo = "";
        roomNo = 0;
        fee = 0;
    }

    void input()
    {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Enter Student Name: ";
        getline(cin, name);

        cout << "Enter Father Name: ";
        getline(cin, fatherName);

        cout << "Enter CNIC: ";
        getline(cin, cnic);

        cout << "Enter Contact Number: ";
```

```
getline(cin, contactNo);


cout << "Enter Student ID: ";
while (!(cin >> studentID))
{
    cout << "Account information does not match. Enter valid Student ID: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}


cout << "Enter Age: ";
while (!(cin >> age))
{
    cout << "Account information does not match. Enter valid Age: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}


cout << "Enter Room Number: ";
while (!(cin >> roomNo))
{
    cout << "Account information does not match. Enter valid Room Number: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

```
    cout << "Enter Hostel Fee: ";
    while (!(cin >> fee))
    {
        cout << "Account information does not match. Enter valid Fee: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int getID()
{
    return studentID;
}

void updateFee(float amount)
{
    fee += amount;
}

void updateRoom(int room)
{
    roomNo = room;
}
```

```
    void display() override
    {
        cout << "\n==================================";
        cout << "\nStudent ID   : " << studentID;
        cout << "\nName         : " << name;
        cout << "\nFather Name  : " << fatherName;
        cout << "\nCNIC         : " << cnic;
        cout << "\nContact No   : " << contactNo;
        cout << "\nAge          : " << age;
        cout << "\nRoom No      : " << roomNo;
        cout << "\nFee Paid     : " << fee;
        cout << "\n==================================\n";
    }
};

// Hostel Class
class Hostel
{
private:
    Student students[100];
    int count;

public:
    Hostel()
    {
```

```
    count = 0;
}

void addStudent()
{
    if (count >= 100)
    {
        cout << "Hostel Record Full!\n";
        return;
    }

    students[count].input();
    count++;

    cout << "\nStudent Added Successfully!\n";
}

void showStudents()
{
    if (count == 0)
    {
        cout << "\nNo Students Found!\n";
        return;
    }
```

```
    for (int i = 0; i < count; i++)
    {
        students[i].display();
    }
}

void searchStudent()
{
    int id;

    cout << "Enter Student ID: ";
    cin >> id;

    bool found = false;

    for (int i = 0; i < count; i++)
    {
        if (students[i].getID() == id)
        {
            students[i].display();
            found = true;
            break;
        }
    }
```

```
    if (!found)
    {
        cout << "\nStudent Not Found!\n";
    }
}

void updateFee()
{
    int id;
    float amount;

    cout << "Enter Student ID: ";
    cin >> id;

    for (int i = 0; i < count; i++)
    {
        if (students[i].getID() == id)
        {
            cout << "Enter Additional Fee: ";
            cin >> amount;

            students[i].updateFee(amount);

            cout << "\nFee Updated Successfully!\n";
            return;
```

```
        }
    }

    cout << "\nStudent Not Found!\n";
}

void updateRoom()
{
    int id, room;

    cout << "Enter Student ID: ";
    cin >> id;

    for (int i = 0; i < count; i++)
    {
        if (students[i].getID() == id)
        {
            cout << "Enter New Room Number: ";
            cin >> room;

            students[i].updateRoom(room);

            cout << "\nRoom Updated Successfully!\n";
            return;
        }
```

```
    }

    cout << "\nStudent Not Found!\n";
}

void deleteStudent()
{
    int id;

    cout << "Enter Student ID to Delete: ";
    cin >> id;

    for (int i = 0; i < count; i++)
    {
        if (students[i].getID() == id)
        {
            for (int j = i; j < count - 1; j++)
            {
                students[j] = students[j + 1];
            }

            count--;

            cout << "\nStudent Deleted Successfully!\n";
            return;
```

```
            }
        }

        cout << "\nStudent Not Found!\n";
    }
};

// Main Function
int main()
{
    Hostel hostel;
    int choice;

    do
    {
        cout << "\n========== HOSTEL MANAGEMENT SYSTEM ==========\n";
        cout << "1. Add Student\n";
        cout << "2. Display All Students\n";
        cout << "3. Search Student\n";
        cout << "4. Update Fee\n";
        cout << "5. Change Room\n";
        cout << "6. Delete Student\n";
        cout << "0. Exit\n";
        cout << "Enter Choice: ";
```

```
cin >> choice;

switch (choice)
{
case 1:
    hostel.addStudent();
    break;

case 2:
    hostel.showStudents();
    break;

case 3:
    hostel.searchStudent();
    break;

case 4:
    hostel.updateFee();
    break;

case 5:
    hostel.updateRoom();
    break;

case 6:
```

```
            hostel.deleteStudent();
            break;

        case 0:
            cout << "\nProgram Ended Successfully.\n";
            break;

        default:
            cout << "\nInvalid Choice!\n";
        }

    } while (choice != 0);
    return 0;
}
```